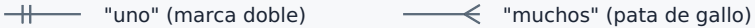
Modelo Entidad-Relación

Entidades, atributos y relaciones de cada microservicio



PK = llave primaria · FK = llave foránea dentro de la misma base · UQ = valor único

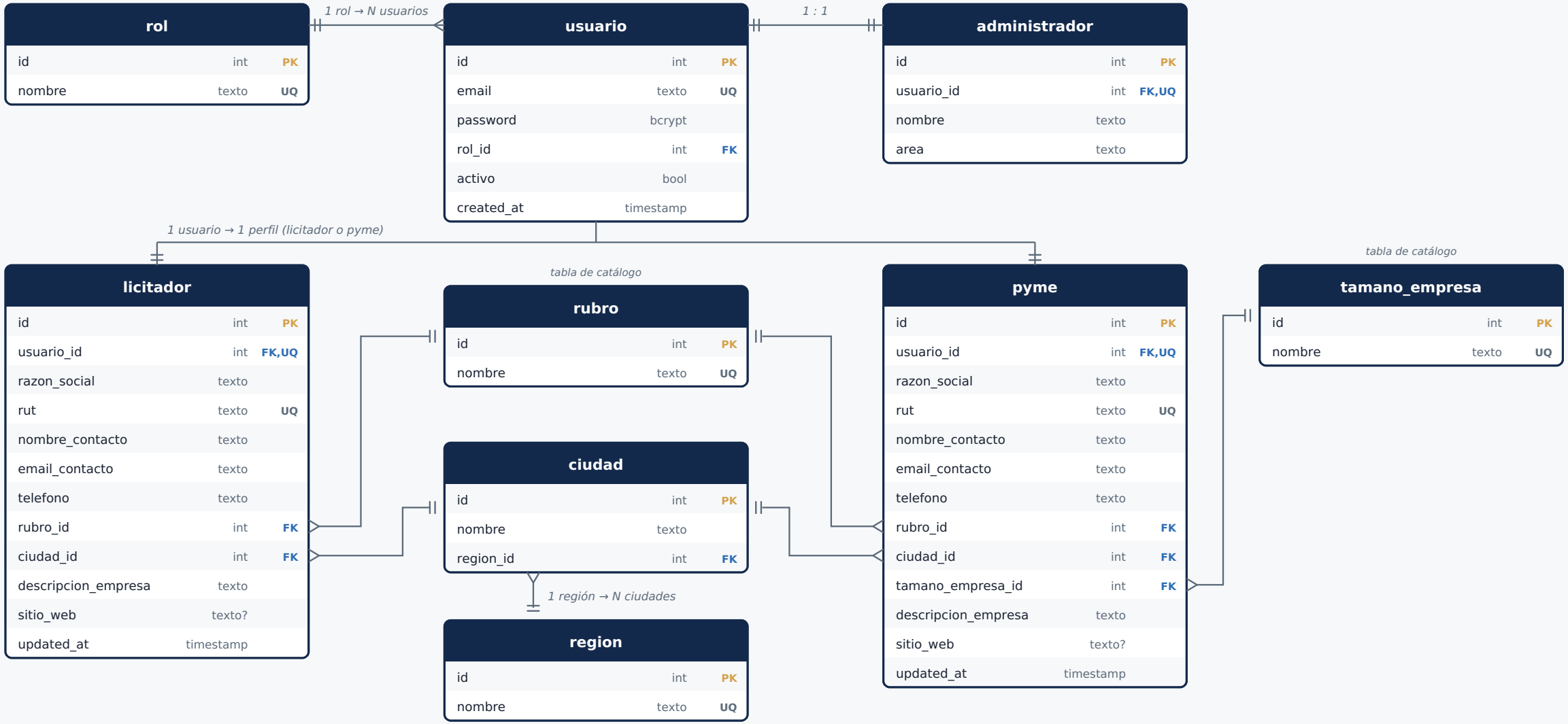
REF = referencia a otro microservicio: no es una FK real (cada microservicio tiene su propia base)
y se valida por llamada REST al microservicio dueño del dato.

Las BD de Usuarios y Licitaciones están normalizadas: rubro, ciudad, estados y tipos son tablas de catálogo.

API USUARIOS — BD normalizada

Perfiles de Licitador y Pyme con catálogos

BD Usuarios: identidad + perfil + tablas de catálogo

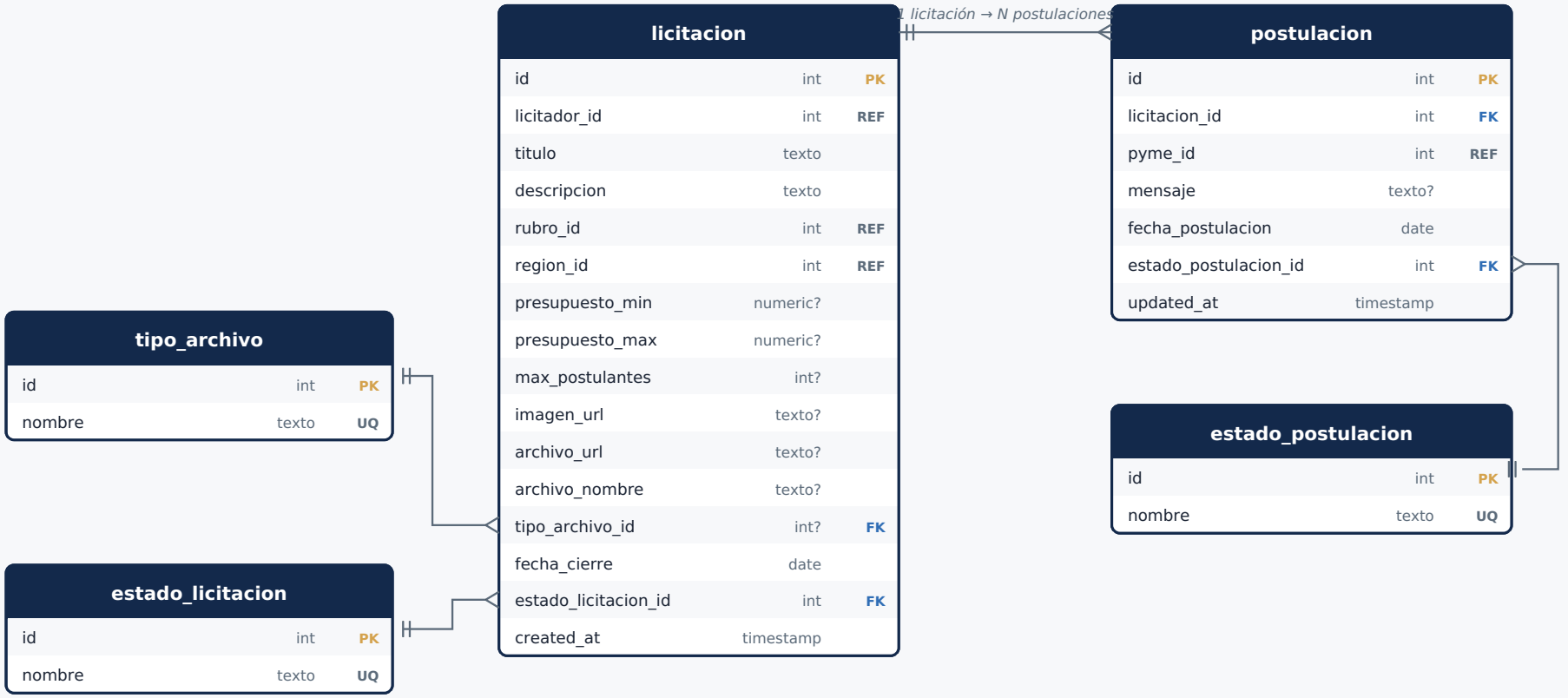


Catálogos: rubro, region, ciudad y tamano_empresa son tablas de referencia. En el formulario de registro se muestran como combobox cargados desde la BD, así Licitadores y Pymes no pueden escribir rubros o ciudades libres (ej. de rubro: Construcción, Tecnología, Salud).
licitador y pyme guardan solo ciudad_id: la región se obtiene desde ciudad.region_id, así el dato no se repite (evita inconsistencias, 3FN).
tamano_empresa solo aplica a pyme (valores: Pequeña, Mediana, Grande). nombre_contacto está en ambos perfiles: es la persona de contacto.
updated_at registra la última edición del perfil: Licitador y Pyme pueden modificar sus datos personales (el RUT y el correo de acceso no son editables).

API LICITACIONES — BD normalizada

Estados y tipo de archivo como catálogos

Relación 1:N — cada licitación puede recibir varias postulaciones



Catálogos de esta base: estado_postulacion (Pendiente, Aprobada, Rechazada), estado_licitacion (Abierta, Cerrada, Adjudicada) y tipo_archivo (PDF, DOCX, XLSX, Otro).
Flujo: la Pyme postula y la postulación nace en 'Pendiente'; el Licitador la pasa a 'Aprobada' o 'Rechazada'. Al aprobar una, la licitación pasa a 'Adjudicada'.
rubro_id y region_id son REF a las tablas rubro y region de la BD Usuarios (catálogos únicos). El filtro GET /api/licitaciones?rubroId=1 usa el mismo id del perfil.
archivo_url / archivo_nombre / tipo_archivo_id: documento complementario opcional del Licitador. En la BD solo se guarda la URL (el archivo va a un storage).
licitador_id y pyme_id no son FKs: son IDs validados por REST contra el microservicio de Usuarios.
Al registrarse una postulación, API Licitaciones llama por REST a API Chat para abrir automáticamente la conversación entre la Pyme y el Licitador.

API MS-VENTAS — BD normalizada

Planes, suscripciones, ventas y pagos

La cadena: plan → suscripción → venta → pago

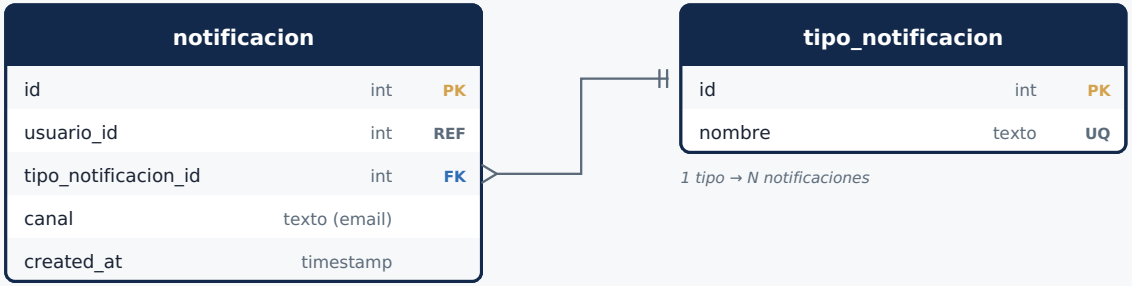


Catálogos nuevos: estado_suscripcion (Activa, Vencida, Cancelada), estado_pago (Aprobado, Rechazado, Pendiente) y metodo_pago (Crédito, Débito). plan_suscripcion ya era un catálogo.
metodo_pago se guarda en pago porque es el medio con que se pagó esa transacción específica (lo informa la pasarela); si se agrega otro medio, solo se inserta una fila.
venta.monto se mantiene aquí a propósito: guarda lo que realmente se cobró ese día, aunque el precio del plan cambie después (dato histórico).
usuario_id en suscripcion no es una FK hacia otra base de datos: es una REF al microservicio de Usuarios y se valida por llamada REST.

API NOTIFICACIONES — BD normalizada

Un registro por cada evento importante

Una tabla principal + un catálogo de tipos de evento



Catálogo nuevo: tipo_notificacion (Licitación publicada, Postulación recibida, Postulación aprobada, Postulación rechazada, Pago confirmado, Mensaje nuevo).
canal queda como texto simple: hoy solo existe 'email', así que un catálogo propio sería sobre-ingeniería; se puede normalizar si se agregan más canales.
No tiene más relaciones: API Licitaciones y API MS-Ventas la llaman por REST cuando ocurre un evento.
API Asistente (LicitAsist) no aparece porque no tiene base de datos propia: solo orquesta llamadas a las otras APIs.

API CHAT — Conversación por postulación

Chat privado entre Licitador y Pyme

Relación 1:N — cada conversación contiene varios mensajes



API nueva con su propia base (bd_chat), igual que Notificaciones: una base por microservicio. No lleva catálogos porque no tiene campos de tipo o estado.
Apertura automática: al registrarse una postulación, API Licitaciones llama por REST a API Chat y crea la conversación (una por postulación: postulacion_id es único).
licitador_id y pyme_id se guardan en la conversación para verificar el acceso en cada mensaje sin llamar a otra API (copia controlada de datos de la postulación).
Seguridad: el gateway valida el JWT y API Chat comprueba que quien escribe o lee sea el Licitador o la Pyme de esa conversación; cualquier otro usuario es rechazado.
Versión MVP: solo texto y consulta periódica cada pocos segundos (polling). WebSocket queda como mejora futura. Mensaje nuevo avisa vía API Notificaciones.
postulacion_id, licitador_id, pyme_id, emisor_id son REF: ids validados por REST contra Licitaciones y Usuarios, no llaves foráneas reales.

FLUJO POST — Registro de Licitador y Pyme

Formulario → API → Persistencia en BD

FLUJO LICITADOR

1 — Formulario de registro Licitador

razon_social, rut, nombre_contacto, email_contacto, telefono, descripcion_empresa, rubro (combobox), región y ciudad (combobox encadenados), email y password.

2 — POST /api/usuarios/registro/licitador

El frontend envía un JSON con los datos del formulario en una sola petición. Los combobox envían ids de catálogo (rubroId, ciudadId).

3 — Inicio de transacción en BD Usuarios

Spring Boot abre una transacción @Transactional: si falla cualquier INSERT se hace rollback completo.

4 — INSERT en tabla 'usuario'

Se guarda email, password (bcrypt), rol_id = id del rol 'licitador', activo = true y created_at = now().

5 — INSERT en tabla 'licitador'

Se guarda usuario_id (FK del paso anterior), razon_social, rut, nombre_contacto, email_contacto, telefono, rubro_id (FK), ciudad_id (FK), descripcion_empresa y sitio_web.

FLUJO PYME

1 — Formulario de registro Pyme

razon_social, rut, nombre_contacto, email_contacto, telefono, descripcion_empresa, rubro (combobox), región y ciudad (combobox encadenados), tamaño (combobox), email y password.

2 — POST /api/usuarios/registro/pyme

El frontend envía un JSON con los datos del formulario en una sola petición. Los combobox envían ids de catálogo (rubroId, ciudadId, tamanoEmpresaId).

3 — Inicio de transacción en BD Usuarios

Spring Boot abre una transacción @Transactional: si falla cualquier INSERT se hace rollback completo.

4 — INSERT en tabla 'usuario'

Se guarda email, password (bcrypt), rol_id = id del rol 'pyme', activo = true y created_at = now().

5 — INSERT en tabla 'pyme'

Se guarda usuario_id (FK del paso anterior), razon_social, rut, nombre_contacto, email_contacto, telefono, rubro_id (FK), ciudad_id (FK), tamano_empresa_id (FK), descripcion_empresa y sitio_web.

Éxito (HTTP 201 Created): se devuelve el perfil completo (usuario + datos de la empresa + nombres de rubro, ciudad y región) para mostrarlo de inmediato.
Error (email o RUT duplicado): HTTP 409 Conflict con mensaje descriptivo; el rollback deshace ambos INSERT.